



现代控制理论 · 个人学习笔记

控制理论 与状态空间

Basstt 著

github.com/BassttElSevic

github.com/BassttElSevic/MyNoteOfControlTheory

目录

第零章 怎么用这套模板记笔记	1
0.1 源文件与成品	1
0.2 编译与查看 (<code>\ll</code>)	1
0.2.1 写笔记的动作只有三步	1
0.2.2 按 <code>\ll</code> 之后发生了什么	1
0.3 vimtex 快捷键与这套配置改了啥	1
0.3.1 vimtex 的按键	1
0.3.2 这套配置在 nvim 里改了些什么	1
0.4 这套模板的文件	2
0.5 LaTeX 基本概念	2
0.6 文字排版	3
0.6.1 效果	3
0.6.2 源码	3
0.6.3 标题层级	3
0.7 列表	3
0.7.1 效果	3
0.7.2 源码	4
0.8 数学公式	4
0.8.1 行内公式	4
0.8.2 独立公式 (带编号)	4
0.8.3 独立公式 (不带编号)	4
0.8.4 多行公式	5
0.8.5 常用符号	5
0.8.6 给公式加标签、引用	5
0.9 图片	5
0.9.1 TikZ: 直接在模板里画	5
0.9.2 TikZ 常用画法 (几种够用的)	6
0.9.3 外部图片	8
0.10 表格	9
0.10.1 效果	9
0.10.2 源码	9
0.11 彩色知识块	9
0.11.1 带编号的知识块	9
0.11.2 不加编号的 <code>notebox</code>	10
0.11.3 自定义一种新颜色、新知识块	10

0.12 插入代码块 (<code>codebox</code>)	11
0.13 如何新增一个章节	11
0.14 我们配好的宏与环境 (重点)	11
0.14.1 带编号的彩色框 (定义/定理/例...)	12
0.14.2 <code>notebox</code> : 不加编号的彩框	12
0.14.3 <code>codebox</code> : 代码块	12
0.14.4 自定义一种新颜色、新框	13
0.14.5 颜色代号对照	13
0.14.6 宏速查表	13
0.15 想改样式、改封面去哪	13
0.16 常用命令速查	13
0.16.1 标题层级	13
0.16.2 文字样式	14
0.16.3 列表	14
0.16.4 数学	14
0.16.5 插图	14
0.16.6 表格	14
0.16.7 引用与文件	14
0.16.8 本模板的环境	14
0.17 用 <code>git</code> / <code>lazygit</code> 管理这份笔记	14
0.17.1 <code>nvim</code> 里分两套工具	14
0.17.2 <code>lazygit</code> 的按键	15
0.17.3 <code>gitsigns</code> 的按键 (改动块)	15
0.17.4 日常提交流程	15
0.18 常见的坑	15
0.19 小结	16

第零章 怎么用这套模板记笔记

这套笔记用 $\text{\LaTeX}$ 写，编译成 PDF。每一节都分两部分：先给效果（这一章里真实渲染出来的样子），再给能复制出这个效果的源码。

0.1 源文件与成品

核心循环

改 `main.tex` → 编译 (`\ll`) → 看 `main.pdf` (`\lv`)。

0.2 编译与查看 (`\ll`)

0.2.1 写笔记的动作只有三步

1. 在 `nvim` 里打开 `main.tex`，按 `\ll` 编译。`\ll` 是连续编译：你保存一次它自动重编一次。
2. 按 `\lv` 打开 `main.pdf` (`Zathura`)，光标跳到你源码所在处。
3. 在 `Zathura` 里 `Ctrl + 左键点 PDF`，跳回源码对应行（反向跳转）。

0.2.2 按 `\ll` 之后发生了什么

- 辅助文件 (`.aux`、`.log`、`.toc`、`.synctex`) 写进 `build/`。
- `main.pdf` 写在项目根目录，紧挨 `main.tex`。
- 目录/引用要靠**编两遍**才正确。

0.3 `vimtex` 快捷键与这套配置改了啥

0.3.1 `vimtex` 的按键

`vimtex` 用 `<localleader>`（这里是 `\`）开头的按键管编译。常用的几个：

按键	作用
<code>\ll</code>	开始/停止连续编译 (<code>latexmk</code>)
<code>\lv</code>	打开 PDF 并跳到光标处 (<code>zathura</code>)
<code>\lc</code>	清理辅助文件
<code>\le</code>	查看编译错误
<code>\lt</code>	章节/目录
<code>\li</code>	编译信息

最常用就是 `\ll`（编译）和 `\lv`（看 PDF）。

0.3.2 这套配置在 `nvim` 里改了什么

配置文件在 `~/.config/nvim/lua/plugins/latex.lua`，主要几处：

- 编译器用 **latexmk**: 自动跑多遍、处理 `\bibliography` 等。
- 默认引擎是 **xelatex**: 为了中文。`pdflatex` 处理不了中文字符;用 `xelatex` 才有 `fontspec` 字库。想单独某个文件用 `pdflatex`, 在文件头写 `% !TEX program = pdflatex`。
- 辅助文件进 **build/**: `aux_dir = "build"`, 源码目录只留 `.tex` 和 `main.pdf`。
- PDF 留在源码旁: `out_dir = ""`, 方便 `\lv`。
- 阅读器用 **zathura_simple**: 支持 SyncTeX 正反向跳转, 且不依赖 X11 的 `xdotool` (本机是 Wayland)。
- 补全和诊断交给 **texlab**(LSP): 关闭了 `vimtex` 自己的补全(`vimtex_complete_enabled=0`)和诊断队列, 避免重复。
- `texlab` 编译手动触发: `onSave = false`, 不每次保存都编(大文档慢), 要编用 `:LspTexlabBuild`。
- `chktex` 静态检查: 打开和保存时跑, 在你写的时候顺手查语法/标点问题。

想改这些

这些都是 `/.config/nvim/lua/plugins/latex.lua` 里的 `vim.g.vimtex_*` 变量, 改那一个文件即可, 跟笔记 `.tex` 无关。

0.4 这套模板的文件

项目结构

```

1 MyNoteOfControlTheory/
2 |— main.tex          # 主文档: 文档类、封面、\include 各章节
3 |— preamble.tex     # 导言区: 字体、配色、页面、知识块宏、代码框宏
4 |— chapters/        # 各章节 .tex
5 |   |— 00_getting_started.tex
6 |— images/          # 外部图片
7 |— build/           # 辅助文件 (自动生成, 已 gitignore)
8 |— main.pdf         # 编译成品 (根目录)
9 |— .gitignore
10 |— README.md

```

你平时只动 `main.tex` 和 `chapters/`。全局样式(字体、颜色、页面、框)去 `preamble.tex` 改。

0.5 LaTeX 基本概念

- 命令: 反斜杠开头, 如 `\section`。
- 参数: 命令可带, 放 `{}` 里, 如 `\textbf{粗体}`。
- 环境: `\begin{名}` 与 `\end{名}` 包住一段内容。
- 注释: `%` 及其后整行不编译。
- 段落: 一个空行 = 分段。

0.6 文字排版

0.6.1 效果

这行加粗。 这行斜体（中文变成楷体）。 这行强调。 这行等宽字体。

0.6.2 源码

文字样式

```
1 \textbf{粗体}           % \textbf
2 \textit{斜体=楷体}      % \textit (中文自动成楷体)
3 \emph{强调}            % \emph
4 \texttt{等宽字体}       % \texttt
```

中文斜体说明

中文没有传统意义上的斜体，按国内习惯用**楷体**代替。这套模板已配好：写 `\textit{某句}`，出来就是楷体，无需额外设置。

0.6.3 标题层级

这一层是 subsection

它是怎么写的

标题命令

```
1 \chapter{章标题}        % 第一章最顶
2 \section{节标题}
3 \subsection{小节标题}
4 \subsubsection{小小节标题}
```

一个.tex 文件只能有一个 `\chapter`，下面可以有很多 `\section` 及更小的标题。编号（第零章、0.1、0.1.1）是自动排的。

0.7 列表

0.7.1 效果

无序列表：

- 第一项
- 第二项

有序列表：

1. 第一步

2. 第二步

0.7.2 源码

列表

```

1 % 无序
2 \begin{itemize}
3   \item 第一项
4   \item 第二项
5 \end{itemize}
6 % 有序
7 \begin{enumerate}
8   \item 第一步
9   \item 第二步
10 \end{enumerate}

```

0.8 数学公式

0.8.1 行内公式

效果：设 $G(s)$ 是传递函数， s 是复变量，输入是 u ，输出是 y 。

源码：`\textbf{设}  $G(s)$  是传递函数。`

0.8.2 独立公式（带编号）

$$G(s) = C(sI - A)^{-1}B \quad (0.1)$$

独立公式（带编号）

```

1 \begin{equation}
2   G(s) = C(sI - A)^{-1}B
3   \label{eq:cs2}
4 \end{equation}

```

`equation` 自动编号，编号是「章. 序」（如 0.1、0.2）。

0.8.3 独立公式（不带编号）

$$G(s) = \frac{Y(s)}{U(s)}$$

源码：`\[ G(s) = \frac{Y(s)}{U(s)} \]`，或用 `\[ ... \]`。

0.8.4 多行公式

$$\dot{x} = Ax + Bu \quad (0.2)$$

$$y = Cx + Du \quad (0.3)$$

多行推导

```
1 \begin{align}
2 \dot{x} &= Ax + Bu \\
3 y &= Cx + Du
4 \end{align}
```

0.8.5 常用符号

这段是渲染出来的： x^n a_1 $\frac{a}{b}$ $\sqrt{x}$ $\sum_{i=1}^n i$ $\int_0^\infty f(t) dt$ α β ω σ Δ

常用符号

```
1 上标 $x^n$      下标 $a_1$
2 分式 \frac{a}{b} 根式 \sqrt{x}
3 求和 \sum_{i=1}^n i 积分 \int_0^\infty f(t) dt
4 希腊 \alpha \beta \omega \sigma \Delta
```

0.8.6 给公式加标签、引用

效果：式 (??) 是状态空间到传递函数的公式。

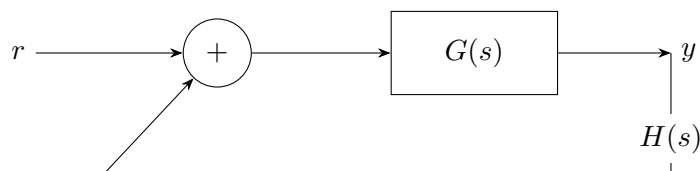
源码：公式里写 `\label{名字}`，别处写 `\eqref{名字}`（自带括号）或 `\ref{名字}`。

0.9 图片

0.9.1 TikZ：直接在模板里画

这套模板推荐用 TikZ 画图，不依赖外部图片文件、可编译、可随文移动。模板已加载 `arrows.meta`、`positioning`、`calc`、`shapes.geometric`。

效果（单位反馈闭环框图）：



上面的框图源码

```
1 \begin{tikzpicture}[>=Stealth, node distance=1.6cm]
2 \node[draw, circle, minimum size=0.9cm] (sum) at (0,0) {$+$};
```

```

3 \node[draw, rectangle, minimum width=2.2cm, minimum height=1.1cm]
4   (plant) at (3.4,0) {$G(s)$};
5 \draw[->] (-2.4,0) node[left] {$r$} -- (sum);
6 \draw[->] (sum) -- (plant);
7 \draw[->] (plant) -- (6.0,0) node[right] {$y$};
8 \draw[->] (6.0,0) -- (6.0,-1.6)
9   node[below, midway, fill=white] {$H(s)$} -| (-1.5,-1.6) -- (sum);
10 \end{tikzpicture}

```

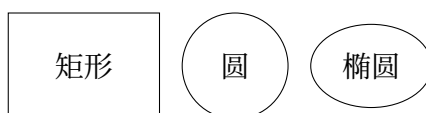
节点命名

每个 `\node` 可起名字 (写在 `()`, 如 `(sum)`、`(plant)`), 后面 `\draw` 直接引用名字连线, 比硬写坐标方便。

0.9.2 TikZ 常用画法 (几种够用的)

基本图形与文字

效果:



源码:

基本图形

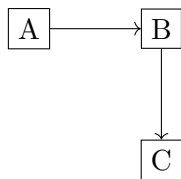
```

1 \begin{tikzpicture}
2   \draw (0,0) rectangle (2,1.4); % 矩形
3   \draw (3,0.7) circle (0.7); % 圆
4   \draw (4.8,0.7) ellipse (0.8 and 0.55); % 椭圆
5   \node at (1,0.7) {矩形}; % 在坐标处放文字
6   \node at (3,0.7) {圆};
7 \end{tikzpicture}

```

相对定位 (positioning 库)

不用手算坐标, 用 `right=of`、`below=of` 让节点互相挨着。效果:



源码:

相对定位

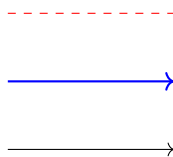
```

1 \begin{tikzpicture}[node distance=1.2cm]
2   \node[draw] (a) {A};
3   \node[draw, right=of a] (b) {B};      % 放在 a 右边
4   \node[draw, below=of b] (c) {C};      % 放在 b 下方
5   \draw[->] (a) -- (b);
6   \draw[->] (b) -- (c);
7 \end{tikzpicture}

```

箭头样式与颜色

效果:



源码:

箭头样式

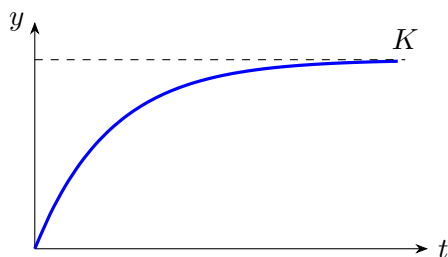
```

1 \draw[->] (0,0) -- (2.2,0);           % 普通箭头
2 \draw[thick,->,blue] (0,0.9) -- (2.2,0.9); % 加粗 + 蓝色
3 \draw[dashed,red] (0,1.8) -- (2.2,1.8); % 虚线 + 红色

```

画函数曲线（一阶阶跃响应）

控制理论常画响应曲线。效果:



源码:

函数曲线

```

1 \begin{tikzpicture}[>=Stealth]
2   \draw[->] (0,0) -- (5.2,0) node[right]{$t$}; % 横轴
3   \draw[->] (0,0) -- (0,3) node[left]{$y$};    % 纵轴
4   % plot 画平滑曲线, domain 是 x 范围, \x 是变量
5   \draw[very thick,blue] plot[smooth,domain=0:4.8] (\x,{2.5*(1-exp(-\x))})

```

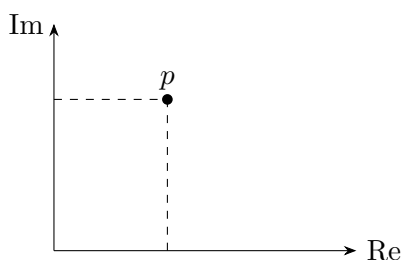
```

;
6 \draw[dashed] (0,2.5) -- (4.9,2.5) node[above]{$K$}; % 渐近线
7 \end{tikzpicture}

```

坐标轴与标注

效果：



源码：

坐标轴与标注

```

1 \begin{tikzpicture}[>=Stealth]
2 \draw[->] (0,0) -- (4,0) node[right]{\operatorname{Re}};
3 \draw[->] (0,0) -- (0,3) node[left]{\operatorname{Im}};
4 \fill (1.5,2) circle (2pt) node[above]{$p$}; % 点 + 标注
5 \draw[dashed] (0,2) -- (1.5,2); % 虚线投影
6 \draw[dashed] (1.5,0) -- (1.5,2);
7 \end{tikzpicture}

```

0.9.3 外部图片

先把图片放进 `images/`，再 `includegraphics` 插入。

插入一张图

```

1 \begin{figure}[htbp]
2 \centering
3 \includegraphics[width=0.6\textwidth]{images/example.png}
4 \caption{图的标题}
5 \label{fig:example}
6 \end{figure}

```

`width=0.6\textwidth` 是图片宽度（正文宽度的六成）。`\caption` 写标题，`\label` 给图起名，之后用 `\ref{名字}` 引用它。

0.10 表格

0.10.1 效果

名称	符号	单位
质量	m	kg
阻尼	c	N s/m

表 1: 一个三线表

0.10.2 源码

三线表

```
1 \begin{tabular}{lcc}
2   \toprule
3   名称 & 符号 & 单位 \\
4   \midrule
5   质量 &  $m$  & kg \\
6   阻尼 &  $c$  & N s/m \\
7   \bottomrule
8 \end{tabular}
```

`lcc` 是列格式 (左对齐、居中、居中)。`\toprule/\midrule/\bottomrule` 是 `booktabs` 的三条横线。

0.11 彩色知识块

所有知识块的样式都在 `preamble.tex` 配好，你只填内容，不写样式参数。

0.11.1 带编号的知识块

写法: `\begin{环境名}{标题}{标签}`。第 1 个参数标题 (可写一对空 `{}` 只显示编号)，第 2 个是供引用的标签。

效果:

定义 0.1 (单边拉普拉斯变换)

对定义在 $t \geq 0$ 的信号，其拉普拉斯变换为

$$\mathcal{L}\{f(t)\} = \int_0^\infty f(t) e^{-st} dt. \tag{0.4}$$

定理 0.1

若系统在原点渐近稳定，则对任意正定 Q ，方程 $A^\top P + PA = -Q$ 有唯一正定解 P 。

例 0.1（一阶系统阶跃响应）

对 $G(s) = \frac{K}{Ts + 1}$ ，单位阶跃响应为 $y(t) = K(1 - e^{-t/T})$ 。

写法：`\begin{环境名}{标题}{标签} ... \end{环境名}`。上面的定义、定理、例就是这么写的。

现有类型与颜色：

环境名	显示名	颜色
definition	定义	蓝
theorem	定理	红
lemma	引理	靛
proposition	命题	紫
corollary	推论	粉
example	例	绿
remark	注	灰

0.11.2 不加编号的 **notebox**

效果：

证明思路

先找 T 的不变子空间，再做三角化。

要点

这是要点框。多行内容之间空一行分段。

写法：`\begin{notebox}[颜色]{标题} ... \end{notebox}`。方括号选颜色（不写默认蓝），花括号是标题。

可选颜色：`bl`（蓝）、`tl`（青）、`gr`（绿）、`og`（橙）、`pu`（紫）、`rd`（红）、`in`（靛）、`pk`（粉）、`gy`（灰）。

0.11.3 自定义一种新颜色、新知识块

效果、源码：

新增一种知识块

```
1 % 在 preamble.tex 里:
2 \definecolor{青}{HTML}{1F9B8E}
3 \newcoloredtheorem{styling}{风格}{青}{sty}
```

注册后就能 `\begin{styling}{标题}{标签}`。第 3 个参数是颜色名，第 4 个是标签前缀。

0.12 插入代码块 (codebox)

codebox 自动加行号、高亮关键字、自动断行。语法: `\begin{codebox} [语言]{标题}`。

效果:

编译命令

```
1 latexmk -xelatex -synctex=1 -auxdir=build main.tex
```

支持多种语言 (bash、latex、text、c、python、matlab 等)，每种都会自动高亮关键字 (蓝)、字符串 (绿)，注释会显示成灰字。

可选语言 (方括号里填名字)

```
1 bash                shell 脚本
2 latex / tex         LaTeX 源码
3 text                无高亮 (目录、说明)
4 C  c / python / matlab
```

0.13 如何新增一个章节

1. 复制 `chapters/00_getting_started.tex` 为 `chapters/NN_ 名字.tex`。
2. 文件首行保留 `% !TEX root = ../main.tex`; 把 `\chapter{名字}` 改成新名。
3. 在 `main.tex` 正文区加一行 `\include{chapters/NN_ 名字}`。

注意 `main.tex` 里已经 `\include` 了第零章，你新写的章节在它下面再加一行即可。

在 main.tex 里登记 (示意新章节加在哪)

```
1 \mainmatter
2 \include{chapters/01_statespace} % 你新加的这行
```

章节顺序由 `\include` 的先后决定。

0.14 我们配好的宏与环境 (重点)

下面这些都是在 `preamble.tex` 里配好的宏/环境，你只填内容，一个样式参数都不用写。

0.14.1 带编号的彩色框（定义/定理/例…）

效果：

命题 0.1（能控性的秩判据）

系统 (A, B) 能控，当且仅当能控性矩阵 $C = [B \ AB \ \dots \ A^{n-1}B]$ 满秩。

推论 0.1

当 C 行满秩时，可以通过状态反馈任意配置极点。

这类框的写法

```
1 % 有标题：{标题}{标签}
2 \begin{proposition}{标题}{标签} 内容 \end{proposition}
3 % 不写标题：只显示编号
4 \begin{corollary}{}{标签} 内容 \end{corollary}
```

可用的名字: definition(定义) theorem(定理) lemma(引理) proposition(命题) corollary(推论) example(例) remark(注)。

0.14.2 notebox：不加编号的彩框

效果：

这一部分解决什么

适合放「这一段要讲什么」「一个要点」「一段提示」。方括号选颜色，不写默认蓝。

notebox 写法

```
1 \begin{notebox}[gr]{标题} 内容 \end{notebox}
2 % 不写颜色=蓝：\begin{notebox}{标题}...\end{notebox}
```

0.14.3 codebox：代码块

效果：

一段 LaTeX 源码

```
1 \begin{equation}\label{eq:demo}
2 F(s) = \mathcal{L}\{f(t)\}
3 \end{equation}
```

写法: `\begin{codebox}[语言]{标题} ... \end{codebox}`。第一个参数是语言, 第二个是标题。

0.14.4 自定义一种新颜色、新框

在 `preamble.tex` 里加两行:

新增一种框

```
1 \definecolor{青}{HTML}{1F9B8E}           % 1. 定义颜色
2 \newcoloredtheorem{styling}{风格}{青}{sty} % 2. 注册成新环境
```

之后就能 `\begin{styling}{标题}{标签}`。`\newcoloredtheorem` 第 1 参数 = 环境名, 第 2 = 显示名, 第 3 = 颜色名, 第 4 = 标签前缀。

0.14.5 颜色代号对照

可用颜色 (notebox 的方括号里填)

```
1 bl= 蓝    tl= 青    gr= 绿    og= 橙    pu= 紫
2 rd= 红    in= 靛    pk= 粉    gy= 灰
```

0.14.6 宏速查表

宏 / 环境	作用	用法
definition 等	带编号彩框	<code>\begin{环境}{标题}{标签}</code>
notebox	无编号彩框	<code>\begin{notebox}[颜色]{标题}</code>
codebox	代码高亮框	<code>\begin{codebox}[语言]{标题}</code>
<code>\newcoloredtheorem</code>	新增一种框	(写在 <code>preamble.tex</code>)

0.15 想改样式、改封面去哪

- 字体 / 配色 / 页面 / 标题字号 / 框样式: 都在 `preamble.tex`。
- 封面: `main.tex` 的 `\begin{titlepage}`, TikZ 精确定位, 改标题、作者、GitHub 链接都在那。

0.16 常用命令速查

这一节把上面用到的命令都列一遍, 方便随时翻。分组如下:

0.16.1 标题层级

`\chapter` (章) / `\section` (节) / `\subsection` (小节) / `\subsubsection` (小小节)。

0.16.2 文字样式

`\textbf{…}` (加粗) / `\textit{…}` (斜体 = 楷体) / `\emph{…}` (强调) / `\texttt{…}` (等宽)。

0.16.3 列表

`itemize` (无序) / `enumerate` (有序), 条目用 `\item`。

0.16.4 数学

行内 `$…$`; 独立公式 `equation` (带编号)、`\[…\]` (不带编号); 多行 `align`; `\frac`、`\sqrt`、`\sum`、`\int`; `\alpha` 等希腊字母。

0.16.5 插图

`includegraphics` (插外部图) / `tikzpicture` (TikZ 画图); 配 `figure` 环境、`\caption`、`\label`。

0.16.6 表格

`tabular`, 三线用 `\toprule` / `\midrule` / `\bottomrule`。

0.16.7 引用与文件

`\label` (起名) / `\ref` / `\eqref` (引用); `\input` (并入) / `\include` (含章节)。

0.16.8 本模板的环境

`definition`、`theorem`、`lemma`、`proposition`、`corollary`、`example`、`remark` (带编号彩框); `notebox` (无编号); `codebox` (代码块); `\newcoloredtheorem` (新增一种)。

0.17 用 git / lazygit 管理这份笔记

这份笔记本身是个 git 仓库, 用来记录每次改动、能回退、能备份到 GitHub。你不需要记很多 git 命令, 用 `lazygit` 这个图形化面板就够了。

0.17.1 nvim 里分两套工具

- `lazygit`: 全屏操作台, 负责提交、看历史、切分支、同步远端。在 `nvim` 里按 `<leader>gg` 打开。
- `gitsigns`: 行内视图, 负责看到哪儿行改了、单块暂存/回滚、行内 `blame`。它贴在编辑器行号旁, 不用单独开面板。

按键	作用	说明
<leader>gg	打开 git 面板	提交、看历史、分支都在这
<leader>gf	当前文件所属仓库	定位到本文件所在的 git 项目
<leader>gl	仓库提交历史	看所有提交
<leader>gc	当前文件提交历史	只看这个文件改过啥

按键	作用	说明
]h / [h	下一个/上一个改动块	跳到你改过的地方
<leader>hs	暂存当前改动块	把这一段加进提交
<leader>hr	回滚当前改动块	撤销这一段
<leader>hS	暂存整个文件	
<leader>hR	回滚整个文件	
<leader>hp	预览改动块	
<leader>hb	当前行 blame	看这行谁改的
<leader>ht	开关行内 blame	
<leader>hd	与索引对比	
<leader>hD	与上次提交对比	

0.17.2 lazygit 的按键

0.17.3 gitsigns 的按键（改动块）

0.17.4 日常提交流程

1. 写完.tex，按 \ll 编译，确认没报错、PDF 正常。
2. 按 <leader>gg 打开 lazygit，看改动列表。
3. 在 lazygit 里：回车看差异 → 空格暂存 → c 输入提交信息 → 回车提交。
4. 需要备份到 GitHub 时，在 lazygit 里按 P (push) 推上去。

记住

这套模板每改一次就编译一次、看一次效果，攒够一个「有意义」的改动（比如新写了一节）就提交一次。提交信息写清楚改了什么，方便以后查历史。

0.18 常见的坑

编译失败先看这里

- \ll 后按 \le 看错误，红色！那行即出错位置。- 中文/特殊字符报错：要用 xelatex（模板默认已用），别用 pdflatex。- \$ 或 {} 不配对：这是最常见的编译错误，数一下有没有闭合。- 图片找不到：确认文件在 images/，路径写对。- 引用显示??：没编两遍，多按几次 \ll。

0.19 小结

文字直接打、要样式用命令；公式用 `$` 或 `equation`；图优先 TikZ、要插图用 `includegraphics`；知识块只填内容（样式在 preamble 配好）；代码用 `codebox`；新章节复制文件加 `\include`。出错先看 `\le`。